実践 Next.js & TypeScript 入門

アカペラバンド運営アプリで学ぶ Web開発の設計と実装完全ガイド

対象読者	新人ソフトウェアエンジニア、Web開発初心者、Next.js/TypeScriptの基礎を固めたい方
テーマ	要件定義から型設計、React Context、集計アルゴリズム、セキュリティ、UI連携の実践
技術スタック	Next.js (App Router), React 19, TypeScript, Tailwind CSS v4

はじめに：本教材の目的

新人のソフトウェアエンジニアが開発現場に出て最も直面する悩みは、「文法は覚えたけれど、実際の要件をもとにどうやって画面やプログラムを組み立てていけばいいのか分からない」という点です。

本教材は、今回実際に開発した「アカペラバンド運営Webアプリ（Acapella Hub）」を題材に、プロのエンジニアがどのような思考プロセスで要件を整理し、型を定義し、アルゴリズムをコードに落とし込んでいるのかを分かりやすく解説した学習資料です。

第1章：モダンWeb開発のアーキテクチャ

1.1 なぜ Next.js と TypeScript を使うのか？

現代のWeb開発においてデファクトスタンダードとなっているのが、Reactベースのフルスタックフレームワークである Next.js と、静的型付け言語の TypeScript です。

・TypeScriptの強み: データの「型（設計図）」を厳密に定義することで、実行前に打ち間違いや型の不一致をコンパイラが検知し、重大なバグを未然に防ぎます。

・Next.jsの強み: 画面表示（フロント）とデータ処理（バックエンド）をシームレスに統合し、App Routerという仕組みでフォルダ構成がそのままWebサイトのURLになります。

1.2 App Routerの「フォルダ＝URL」の仕組み

ファイルパス	ブラウザのURL	画面の役割
src/app/page.tsx	/	ダッシュボード（トップ画面）
src/app/events/page.tsx	/events	日程調整一覧画面
src/app/events/[id]/page.tsx	/events/ev-1 など	詳細画面（[id]は動的パラメータ）
src/app/events/[id]/respond/page.tsx	/events/ev-1/respond	回答入力画面（受付中のみ利用可）
src/app/events/[id]/vote/page.tsx	/events/ev-1/vote	最終投票 & 自動確定画面
src/app/locations/page.tsx	/locations	練習場所一覧 & Google Map埋め込み
src/app/admin/users/page.tsx	/admin/users	管理者専用メンバー追加画面

第2章：要件定義からデータ構造への落とし込み

2.1 「データの最小化原則」の実践

開発初期、ユーザーから「セキュリティを考慮し、個人情報は絶対に持たせたくない」という重要な要望がありました。Webセキュリティの基本は『持っていないデータは絶対に漏洩しない』ことです。

氏名やメールアドレス、パートなどを排除し、システムに必須の id, password, nickname, bio, isAdmin のみに厳選しました。

```
// src/types/index.ts
export interface User {
  id: string;        // メンバー固有ID (例: "ken")
  password: string;  // ログインパスワード
  nickname: string;  // あだ名・バンド内呼称
  bio?: string;      // 自己紹介 (任意)
  isAdmin?: boolean; // 管理者権限フラグ (メンバー追加権限)
}
```

2.2 進行状態の厳格な型管理（Union Type）

```
export type EventStatus =
  | "open"        // 回答受付中（メンバーが回答可能）
  | "aggregated"  // 集計完了（条件一致候補0件）
  | "voting"      // 最終投票中（複数候補から投票）
  | "confirmed"   // 確定済み（日時・場所決定）
  | "completed"   // 練習終了（過去の記録としてアーカイブ）
  | "cancelled";  // 中止
```

第3章：状態管理と認証ガード（AuthGuard）

3.1 React Contextによる記憶部屋の作成

コンポーネント間でデータを受け渡す際、親から子、孫へと延々とバケツリレーするのは保守性が落ちます。そこで AppContext という共有の記憶部屋を作り、ブラウザの localStorage
と同期させることで、F5再読み込みしてもデータが消えない仕組みを作りました。

3.2 認証ガード（未ログイン時の遮断）

ログインしていないユーザーがアプリの画面を一切操作できないようにする「認証ガード」は、ルートレイアウトで currentUser === null を判定してログイン画面を割り込ませる設計を採用しています。

```
// src/components/ClientProviders.tsx
function AuthContent({ children }: { children: React.ReactNode }) {
  const { currentUser } = useApp();
  // 未ログインなら問答無用でログイン画面を返す！
  if (!currentUser) return <LoginForm />;
  return <><Header /><main>{children}</main></>;
}
```

第4章：集計・投票アルゴリズムの徹底解説

4.1 30分スロット分割とスライディング探索

「指定した練習時間を満たし、共通のスタジオが空いている最大人数の枠」を見つけるため、時間を30分単位のコマに分解し、練習時間の長さを持つウィンドウを朝から夜まで30分ずつスライドさせて走査します。

4.2 練習時間最大化と民主的な確定ロジック

日程調整の確定アルゴリズムには、バンド練習を最大化するための以下の3段階の判定基準を実装しました：

1. 最も多くの人が参加可能: 総参加可能人数（フル＋途中参加）が最大の候補を抽出する。
2. 最も多くの時間練習が可能:
人数が同じ場合、全メンバーの合計参加可能時間（totalAttendanceMinutes）が最大の候補を抽出する。
3. フル参加メンバーが最多: 時間も同じ場合、遅刻・早退のないフル参加者が最も多い候補を抽出する。
4. 自動確定 vs 最終投票:
上記で単独1件に絞り込まれれば直ちに日程確定！同条件で複数残った場合のみ最終投票へ進む。
5. 同票時の幹事決定: 最終投票で同票となった場合は、同票候補の中から幹事が最終決定を行う。

4.3 途中参加・途中退出（遅刻・早退）の区間重なり判定アルゴリズム

バンド練習では「2時間枠のうち後半90分だけ参加できる」「前半だけ参加して早退する」といった現実的なケースが頻繁に起こります。そこで、候補枠 [startM, endM] とメンバー回答 [slotStart, slotEnd]
の時間の重なり（Overlap）を計算し、30分以上参加できる場合は途中参加として救済するロジックを導入しました：

- フル参加 (full): 枠全体に参加可能（slotStart ≤ startM かつ slotEnd ≥ endM）
- 遅刻参加 (late): 開始に遅れるが終了まで参加（slotStart > startM かつ slotEnd ≥ endM）
- 早退 (early): 開始から参加できるが途中で抜ける（slotStart ≤ startM かつ slotEnd < endM）
- ランキング順位:
総人数（フル＋途中）が多い候補を優先し、人数が同じなら「フル参加人数が多い候補」を上位にする。

4.4 単体テスト（Vitest）による品質担保と現場の流儀

複雑な集計アルゴリズムやタイブレーク判定を画面のポチポチ操作だけで手動テストするのは、ミスや見落としの原因になります。本アプリでは vitest を導入し、集計・時刻変換・タイブレーク・認証の全25ケースをわずか0.3秒で全自動検証する単体テスト環境を構築しました。コードを変更した瞬間に「既存の機能が壊れていないか」を一瞬で保証できるのがテスト駆動の強みです。

第5章：現場で役立つUI/UXテクニック

5.1 Google Map との連動（iframeとURLパラメータ）

練習場所一覧では Google Maps の Embed URL を動的に生成して地図を埋め込んでいます。また、日程詳細から /locations?select=loc-1 と遷移することで、該当施設を自動で選択・地図表示する直感的なUXを実現しています。

5.2 練習終了（completed）ステータスの自動遷移

確定した練習日程の終了時刻を過ぎると、バックグラウンドで自動的に「練習終了」ステータスに切り替わり、ダッシュボードの「過去の記録」へアーカイブされます。これにより、将来の振り返りチャットや音源共有への布石が整いました。

第6章：新人エンジニアがつまずきやすいトラブル解説

6.1 「npm コマンドが認識されない」環境変数PATHの仕組み

Node.jsをインストールしても、すでに開いていたターミナルは古い環境変数PATHを保持しているため認識されません。ターミナルを開き直すことで最新の検索リストが読み込まれ、解決します。

6.2 「デザインが当たらない」CSSビルドチェーンの重要性

Next.js 16 と Tailwind CSS v4 の連携において、直接ローダーに依存せず @tailwindcss/cli で確実に静的CSSを事前ビルドするチェーン（npm run build:css && next dev）を組むことで、確実で安定したスタイリングを実現しました。

おわりに：プロのエンジニアへの第一歩

Webアプリ開発の本質は、『ユーザーの要求や運用ルールを深く理解し、破綻のないデータ構造と使いやすい画面に翻訳すること』です。本アプリに組み込まれたコードを何度も読み返し、自分なりに改造してみることで、確実に本物の開発力が身につきます。これからの成長を心より応援しています！